

Elastic Algorithms for Guaranteeing Quality Monotonicity in Big Data Mining

Rui Han¹, Lei Nie^{1,2,#}, Moustafa M. Ghanem¹, Yike Guo^{1,*}

¹Imperial College London, London, UK

²Institute of Computing Technology, Chinese Academy of Sciences, Beijing, China
{r.han10, l.nie, mmg, y.guo}@imperial.ac.uk

Abstract— When mining large data volumes in big data applications users are typically willing to use algorithms that produce acceptable approximate results satisfying the given resource and time constraints. Two key challenges arise when designing such algorithms. The first relates to reasoning about tradeoffs between the quality of data mining output, e.g. prediction accuracy for classification tasks and available resource and time budgets. The second is organizing the computation of the algorithm to guarantee producing better quality of results as more budget is used. Little work has addressed these two challenges together in a generic way. In this paper, we propose a novel framework for developing elastic big data mining algorithms. Based on Shannon’s entropy, an information-theoretic approach is introduced to reason about how result quality is affected by the allocated budget. This is then used to guide the development of algorithms that adapt to the available time budgets while guaranteeing producing better quality results as more budgets are used. We demonstrate the application of the framework by developing elastic k-Nearest Neighbour (kNN) classification and collaborative filtering (CF) recommendation algorithms as two examples. The core of both elastic algorithms is to use a naïve kNN classification or CF algorithm over R-tree data structures that successively approximate the entire datasets. Experimental evaluation was performed using prediction accuracy as quality metric on real datasets. The results show that elastic mining algorithms indeed produce results with consistent increase in observable qualities, i.e., prediction accuracy, in practice.

Keywords—elastic data mining algorithms; quality monotonicity; entropy; R-tree

I. INTRODUCTION

A. Context and Motivation

A key practical challenge facing the analysis of large data volumes in many applications is that the computation is always required to be conducted under varying resource constraint and time constraints. To deal with such problems, users are usually willing to accept a useful approximate result from their computation that can be produced under the available budget (whether time or resources). In the context of data mining tasks, approximate results can be produced typically either by restricting the size of the input data fed to exact algorithms, or by using approximating algorithms over full datasets. Within this context, it is appealing to organize the data mining process to support a “Pay-as-You-Go” model, where users are guaranteed not only a useful approximate result on a tight

budget but also guaranteed results of better quality if more budget is used. We call algorithms that meet this property *elastic algorithms*. Informally, an elastic algorithm is one that takes a dynamically changing time (or resource) budget as input and produces a result whose quality, based on some metric, improves monotonically with the budget used.

Several issues need to be addressed when developing budget-aware data mining algorithms. The first is being able to define and use a *quality measure* to evaluate how good the approximate results are. The second is being able to reason effectively about how using the budget affects the quality of results. The third, and more challenging issue, is to design the data mining algorithm itself so that the quality measure improves as more budget is used. This monotonic improvement should indeed bring consistent increase in observable quality, such as prediction accuracy in classification or recommendation algorithms, as more budget is used.

Various paradigms exist for developing time-adaptive algorithms that operate under time and budget constraints. For example, a wide class of incremental learning algorithms [1, 2, 3] can be used when an entire dataset is not accessible or cannot be processed under limited resources, e.g. memory. Another wide class of algorithms is known as anytime algorithms [4, 5, 6]. These are interruptible algorithms that are designed to return a valid result whenever the algorithm is interrupted, e.g. when the time allocated for the computation expires. However, little work has been conducted towards guaranteeing that by conducting more computation (i.e. using more budget) the result quality would indeed improve.

B. Contributions

In this paper, we propose a generic approach for developing elastic data mining algorithms that guarantee quality monotonicity with respect to allocated time budgets. The concrete contributions of the presented work are:

A generic framework for designing quality-monotonic elastic algorithms: We present a generic framework for designing quality-monotonic elastic algorithms. Such algorithms are suitable for a wide class of data mining problems that are based on finding an approximate result by solving an optimisation problem over a dataset. Our framework for designing such algorithms comprises two components: a *coding component* and a *mining component*. The coding component applies compression techniques to proactively map a dataset into a set of codes with smaller lengths, i.e., ones that require shorter processing time. By processing a code with suitable length, the mining component can then produce a useful approximate result within a specified time budget. We define two properties for these two components, *entropy-*

* Corresponding author

This author is supported by the Strategic Priority Program of Chinese Academy of Sciences (Grant XDA06010400).

monotonicity for the coding component and *entropy-preservation* for the mining component. We provide proofs that any data mining algorithm whose components satisfy both properties guarantees quality monotonicity.

Case studies of developing two elastic mining algorithms: We demonstrate the validity of our approach by designing two elastic mining algorithms. The algorithms use standard naïve k-Nearest-Neighbour (kNN) classification [7] or neighbourhood-based collaborative filtering (CF) [8] over an R-tree coding component [9]. The codes produced by the R-tree are the nodes at different depths that successively approximate the training set at different levels of granularity. We present experimental evaluation on real datasets to demonstrate that both algorithms exhibit consistent increase in observable quality, i.e., prediction accuracy, in practice.

C. Paper Layout

The paper is organized as follows. Section II reviews related work and discusses its relationship with our work. In Section III, we formulate the elastic mining algorithms targeted by this paper by formally defining their quality measure, two components and associated properties. In Section IV and V, the elastic kNN and CF algorithms are introduced respectively. Finally, we conclude the paper in Section VI.

II. DISCUSSING RELATIONSHIP TO RELATED WORK

At present, many techniques have been developed to produce approximate results under resource constraints by restricting data size or computations.

Incremental learning is a well-known methodology for machine learning under resource constraints. The basic idea of incremental learning algorithms is to sequentially process more data to continually update existing acquired knowledge. An incremental learning algorithm benefits from new raw data and integrates it with previously acquired data to refine a learning result. Many incremental learning algorithms such as SVM [1], decision tree [2] and k-means clustering [3] have been proposed. In general, most of existing incremental learning algorithms apply a simple data accumulation strategy (e.g., random sampling from entire data) to accept new data. This means guaranteeing quality monotonicity of results is difficult.

Anytime algorithms provide a generic approach for producing approximate results within time constraints. The basic idea of an anytime algorithm is that once an initial approximate result is produced, the algorithm can be interrupted to output a result at anytime. If the algorithm is not interrupted it continues to incrementally improve its result [4]. The use of the framework has led to implementations of anytime of k-means clustering [5] and kNN classification [6]. In general, anytime algorithms can hide quality decrease by storing and returning the best result obtained so far. They, thus, do not guarantee effective use of extra budget.

Discussing relationships: The approaches discussed above focus on producing approximate results under specific constraints, such as inaccessibility of data or insufficiency of resources in *incremental learning algorithms*, and the requirement of returning a valid result when interrupted in *anytime algorithms*. Little work has been done in such methodologies towards guaranteeing quality monotonicity with respect to time (or resource) budget. Our proposed approach

offers a promising methodology for these paradigms by providing a systematic way to guaranteeing such monotonicity.

III. ELASTIC DATA MINING ALGORITHMS

A. Overview

Informally, we define an elastic mining algorithm as one that produces a result whose quality, based on some metric, improves monotonically with the allocated time budget. Specifically, an elastic algorithm can adapt to a given time budget to produce an approximate result. If extra budget is allocated, the algorithm can build on the acquired result to produce a refined result with improved quality. In this sense, the key property of elastic algorithms is that the quality of results is elastic with respect to the computations. However, in practice, many data mining algorithms do not have such natural structure encapsulating the inherent elasticity. In this section, we provide a formal definition of a theoretical quality measure that can be used for a wide variety of data mining tasks. We then provide a more formal definition of elastic data mining algorithms. We then describe the conditions that must be met by any data mining algorithm to guarantee improving quality with more computations.

B. Definition of Quality Measure

We can formulate many data mining problems (tasks) as optimisation problems. Assume that a dataset S to be mined is drawn randomly from a prior distribution $S \sim \text{prior}(s)$. Given an objective function $f(S, x)$ where the optimisation variable $x \in \mathcal{X}$, a data mining problem aims at finding a value of x in its domain $\mathcal{X}$ that maximises the value of optimisation function. We call the value of x that gives the maximal value of objective function as the **Optimal Result (OR)**:

$$OR = \operatorname{argmax}_{x \in \mathcal{X}} f(S, x)$$

Given a fixed objective function f and a domain $\mathcal{X}$ in a data mining problem, we define OR as the function of dataset S , denoted as $OR(S)$. Thus, an OR is a random variable depending on variable S . For example, in the kNN classification problem, the target is to find the k nearest training points of a test point q to predict its class label. Thus, S is the dataset to be searched, objective function f is the total distance between these points and q , and the OR is q 's k closest points in S .

The execution of an elastic data mining algorithm is constrained by the specified time budget. Given a fixed time budget b and a dataset S , the above optimisation problem can be solved approximately to produce a useful result. We define such result as **Approximate Result: $AR(S, b)$** . Note that similar to an OR , an AR is a function of dataset S and time budget b is also a random variable.

Assuming the prior distribution $\text{prior}(s)$ of dataset S is known, the prior distribution of the optimal result $OR(S)$ is fixed. The uncertainty of an OR can be described as the entropy $H(OR)$ of the OR . In other words, we have to obtain at least $H(OR)$ bits of information to recover a value of OR exactly.

Given an algorithm and a value ar of AR produced by the algorithm, we can calculate the conditional entropy $H(OR|AR = ar)$. This entropy represents the number of bits

required to recover the value of OR exactly given an observed value ar . Thus, we can use the difference between $H(OR)$ and $H(OR|AR = ar)$, known as information gain [10, 11], to measure of the quality of value ar . This quality can be explained as the benefit of the value ar , i.e., the decreased number of bits for exactly recovering the value of OR from a prior state to a state that takes a given value ar .

Definition 1 (Quality measure of a value ar of AR): Given a value ar of AR , its quality is defined as follows:

$$Q(ar) = H(OR) - H(OR|AR = ar)$$

C. Definition of Elastic Data Mining Algorithms

A data mining process can be viewed as a process of two stages. At the first stage, an observation dataset is coded using an assumed base (representation) for the following mining. Coding components range from transformation (such as wavelets based transformation) or other feature based representation (feature extraction) techniques. The coded data is then inputted into a mining component to solve a specific data mining problems/tasks, such as classification or recommendations. The mining component outputs some results that are meaningful to users such predicted class labels or predicted values of test points. In elastic mining scenario, a large-scale dataset needs to be processed within a limited time (resource) budget to produce some useful approximate result. The key idea to solve this challenge is to represent the observational dataset in a proper hierarchical base using the coding component so that the mining can be elastic with respect to the time budget, as shown in Figure 1.

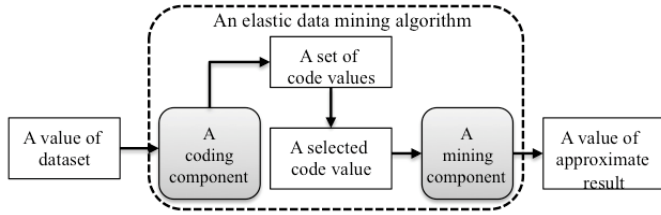


Fig. 1. The two components of an elastic algorithm

A **coding component** is a map from the space of the dataset to a code space. A coding component is lossy if the corresponding map is not injective. The basic idea of lossy coding is to preserve the important information of dataset and remove unimportant parts. The remaining information is then organised using a suitable data structure. A code $C(S, l)$ is the output of a coding component and is a function of dataset S and **length budget** l , where l denotes the maximal length of a code value that can be processed by the elastic algorithm within the time budget b . Thus, a code is a variable depending on variable S and l . Given a fixed time budget b and fixed resources, a fixed length budget can be estimated. A **code value** is selected such that it has the *maximum* length value smaller than the length budget. Note that the coding component is only applied once for a value s of dataset to produce a set of code values with various lengths to be used in the elastic algorithm.

Typically, there are two ways of performing compression of a particular dataset s : instance reduction and feature reduction. In instance reduction, the number of data points in s is reduced. A typical component to index these points in a hierarchy using tree data structures such as quadtree [12], cone

tree [13], and R-tree [14]. At different levels of the tree, the information of the data points can be statistically summarised. In feature reduction, the feature number of data points is decreased. Many feature reduction techniques such as wavelet, principal component analysis (PCA) and hashing functions [15] can be applied in this type of compression.

A **mining component** M is designed to solve a specific data mining problem. A mining component $M(C)$ needs to take a code C as input and output an approximate result. Given a mining component and a code value c , we assume that the running time of the mining component can be primarily determined by the length of the code value, and this running time should be smaller than or equal to the given time budget b . Hence, the bound of an elastic data mining algorithm's running time can be controlled by tailoring the length of the inputted code value.

D. Properties of Quality-Monotonic Elastic Algorithm

We first define the resolution measure of a code value. Since the prior distribution $prior(s)$ of dataset S is known, the uncertainty of S can be described as the entropy $H(S)$ of the S , i.e., the minimal number of $H(S)$ bits information needed to recover the value of S exactly. Given a coding component and a code value c produced by this coding component, the conditional entropy $H(S|C = c)$ represents the number of bits required to recover the value of S exactly given an observed code value. Similar to the quality measure, we use information gain, namely the difference of $H(S)$ and $H(S|C = c)$, to measure of the resolution of the code value c .

Definition 2 (Resolution measure for a code value c): Given a value c of code C , its resolution is defined as follows:

$$R(c) = H(S) - H(S|C = c)$$

To guarantee quality monotonicity, we need a coding component producing codes whose resolution is increasing with their length budget.

Definition 3 (Entropy-monotonicity of a coding component): Fixing a coding component and a value s of dataset S , a coding component is entropy-monotonic if for any two length budgets $l \leq l'$, $H(S|C = C(S = s, l)) \geq H(S|C = C(S = s, l'))$, equivalently, $R(C(S = s, l)) \leq R(C(S = s, l'))$

The entropy-monotonicity property can be explained simply as follows: if a larger budget is given, a coding component will output a more accurate code value. Furthermore, we define a property of the mining component: if the input code value is of higher resolution, then the output of the mining component will have a higher value of quality.

Definition 4 (Entropy-preserving mining component): Let c and c' be any two code values. Assume that $H(S|C = c) \geq H(S|C = c')$ (equivalently, $R(c) \leq R(c')$). A mining component M is entropy-preserving if $H(OR|AR = M(c)) \geq H(OR|AR = M(c'))$, equivalently,

$$Q(M(c)) \leq Q(M(c'))$$

Satisfying these two properties, the quality monotonicity of a data mining algorithm is guaranteed.

Theorem 1: Let a data mining algorithm consist of an entropy-monotonic coding component and an entropy-preserving mining component. Fixing a value of dataset, the quality of

approximate value outputted by this algorithm increases monotonically with the time budget.

Proof. Given a value s of dataset S , a set of corresponding code values with various length budgets is fixed. Given two time budgets $b \leq b'$, let c and c' be two code values selected according to b and b' . We have the length of c is shorter than the length of c' . The coding component is entropy-monotonic, $R(c) \leq R(c')$ according to Definition 3. Using the entropy-preserving mining component to process the two code values c and c' , the corresponding two values of approximate results will satisfy $Q(M(c)) \leq Q(M(c'))$ according to Definition 4. ■

IV. EXAMPLE 1: AN ELASTIC KNN ALGORITHM

Based on the above generic framework, we develop an example elastic kNN algorithm. The core of the algorithms is to use standard naïve kNN classification (i.e., mining component) [7] over an R-tree coding component [9]. The codes produced by the R-tree are the nodes at different depths that successively approximate the training set at different levels of granularity. In this section, we introduce the two components of the algorithm in Section IV.A and Section IV.B, and explain how an approximate result is produced in Section IV.C. Finally, we experimentally evaluate the elastic kNN algorithm in Section IV.D.

Note that due to the limit of space, the proofs that the R-tree coding component satisfies the property of entropy-monotonicity and the kNN mining component has the property of entropy-preservation are not presented. In addition, since no random variable is discussed in this section, we omit “value” for simplicity. For example, a code c denotes a specific code and an approximate result ar represents a specific result.

A. The R-tree Coding Component

In an R-tree, a node consists of a set of n entries $\{e_1, \dots, e_n\}$ where $n \geq 1$. In a leaf node, each entry e refers to a data point z . In a non-leaf node, each entry e refers to one of its child nodes. Each R-tree nodes has a *Minimal Bounding Rectangle (MBR)*, which is the minimal bounding rectangle bounds its corresponding data points. Given a dataset s , the R-tree coding component indexes all the data points in a set s in a hierarchical way by grouping points with similar attribute values to the same node in order to preserve data similarity. For example, Figure 2(a) shows a dataset s consisting of 12 data points in two-dimensional space and Figure 2(b) shows an R-tree constructed to index these points with three levels (depths). It can be observed that at depth 0 of the R-tree, the root node represents all 12 points at the coarsest level of granularity. In contrast, the nodes at depth 2 only enclose two or three points. Hence these nodes represent set s at a finer level of granularity.

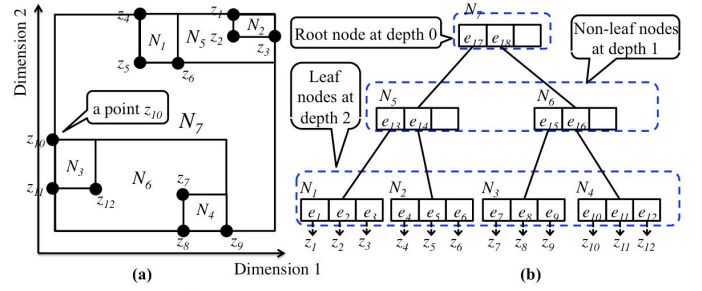


Fig. 2. An Example R-tree Coding

For simplicity, we discuss binary kNN classifier with positive class c_P and negative class c_N and our result can be extended to multiple classes. Thus, given a training set, the R-tree coding component indexes all training points hierarchically using *two R-trees*. One positive R-tree indexes training points from the positive class and another negative R-tree indexes training points from the negative class. Thus, *one R-tree node* summarizes a set of training points from the *same* class. The output of the R-tree coding component is a set of codes, where each **code** corresponds to all the nodes at one depth of the two R-trees. These nodes represent the dataset, at specific granularity, to be used as input to the mining component.

B. The Naïve kNN Classification Algorithm

The naïve kNN classification algorithm [7] classifies a test point q by linearly scanning all data points in the training set with known class labels and finding the k ones whose distances are the closest to q as its k nearest neighbours. The algorithm then decides q 's class label according to the majority vote of the k nearest neighbours, i.e., assigning q to the class with more nearest neighbours.

C. Calculation of an Approximate Result

Before delivering classification services, the elastic kNN algorithm first applies the R-tree coding component to take a training set and generates a set of codes. Once a time budget is given, the algorithm estimates the *length budget* of code, i.e., the maximal number of nodes that can be linearly scanned by the naïve kNN classification method within the time budget. A code c is then selected that it has the maximum length value smaller than the length budget. Using c and a test point q , an approximate result ar can be produced, which is the k nodes in the code that have the smallest distances to q . Using these k nodes, q 's class label can be predicted.

D. Experimental Evaluation

In this section, we evaluated the property of quality monotonicity by producing a list of approximate results and demonstrated how their qualities gradually improve when more computations were conducted. The *prediction accuracy* is applied as the quality measure. Briefly, in a binary kNN classifier, the *prediction accuracy* denotes the proportion of test points that are correctly classified. We also discussed the connection between an approximate result's theoretical quality measure (information gain) and observable quality measure (prediction accuracy).

Experimental platform and datasets: We ran all the experiments on a Linux based workstation with four Intel Core i7 processors with a speed of 2 GHz and 4GB of RAM. Two

large datasets selected from the UCI machine learning repository [16] were tested: the *skin* dataset with 245,057 three-dimensional data points and the *gas* dataset with 13,910 128-dimensional points. To provide a common benchmark to assess different datasets, we randomly removed 100 points from both datasets to form their test sets and leave the remaining points to form the training set.

Evaluation settings: In our experiments, we constructed two R-trees for each dataset that can provide five code values c_1 to c_5 , which corresponds to nodes at depth 1 to 5 of the R-trees and code value c_5 consists of leaf nodes. The R-tree construction time is 46.65 and 119.86 seconds for the *skin* and *gas* datasets, respectively. In all evaluation, we set the value of nearest neighbours $k=5$ and produced five approximate results ar_1 to ar_5 using code values c_1 to c_5 .

Prediction accuracy of optimal result. For both dataset, we first used the naïve kNN algorithm to linearly scan all the training points to produce the *optimal result*. The prediction accuracies of the optimal result for the *skin* and *gas* datasets are 1.00 and 0.92, respectively.

Discussion on the connection between an approximate result's theoretical quality measure (information gain) and observable quality measure (prediction accuracy). Since the elastic kNN algorithm guarantees quality monotonicity: the five approximate results ar_1 to ar_5 have decreasing information gain, i.e., increasing level of approximation to the optimal result. Hence, the observable qualities of these results should gradually approach the qualities of the optimal result. Figure 3 shows the experiment results, where the x axis lists computational time of producing results ar_1 to ar_5 and the y axis displays their prediction accuracies. The results display that the elastic kNN algorithm guarantees the monotonicity of quality in all cases: once users conduct more computations, they can get a result with higher accuracy. We can also observe that these values stepwise approach the accuracy of the optimal result.

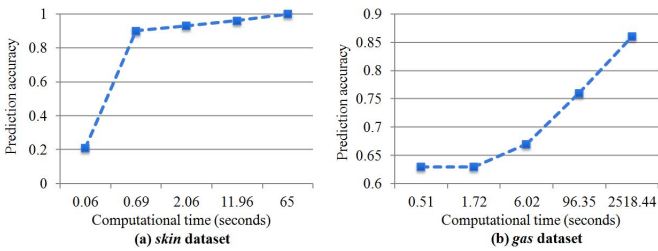


Fig. 3. Experimental result of the elastic kNN algorithm on the *skin* and *gas* datasets.

V. EXAMPLE 2: THE ELASTIC CF ALGORITHM

In this section, we introduce another elastic collaborative filtering (CF) algorithm that applies the R-tree data structures as the coding component and the basic neighbourhood-based CF as the mining component.

A. The Basic Neighbourhood-based CF Algorithm

Neighbourhood-based CF is one predominant type of CF algorithms that has been applied in many recommendation systems such E-commerce websites [8]. Formally, given a

training set of $m \times n$ user-item rating matrix with m users and n items, the entry $r_{v,j}$ denotes user v 's rating on item j ($1 \leq v \leq m$ and $1 \leq j \leq n$). The neighbourhood-based CF algorithm has two major phases. Given an active user u 's target item i to be rated, at the first phase, the algorithm calculates the similarity or weight $w(u, v)$ between user u and any other user v who has rated item i in the matrix. The Pearson's correlation coefficient is used as the weight metric in this work. At the second phase, the algorithm generates the prediction $p(u, i)$ of user u 's rating on item i by taking a weighted average of all ratings on item i from user u 's neighborhood users, $p(u, i) = \bar{r}_u + \frac{\sum_{v \in U} w(u, v) \times (r_{v,i} - \bar{r}_v)}{\sum_{v \in U} |w(u, v)|}$, where I is the set of users that have rated item i , $|w(u, v)|$ denotes the absolute value of weight $w(u, v)$.

B. The Two Components in the Elastic CF Algorithm

The elastic CF algorithm first applies feature reduction using incremental SVD [17] to generate a reduced feature matrix of the user-item space, where each user is represented by a vector of a small number of features. The R-tree coding component then hierarchically indexes the user vectors in this reduced where each node summarizes a group of users. All nodes at a given depth of the R-tree represent a code value, based on the original user-item matrix[18], that can be used by the mining component.

Since the neighbourhood-based CF needs users' ratings on different items and their average ratings to generate prediction, each R-tree node is extended to store aggregated rating information of the users enclosed by this node from the original user-item rating matrix. Specifically, in an R-tree node N whose MBR encloses a set U of users' aggregated ratings. For each item i , suppose a subset U_i of users have rated i ($U_i \subseteq U$), node N 's rating $r_{N,i}$ on i and average rating $\bar{r}_{N,i}$ depending on i can be calculated as: $r_{N,i} = \sum_{u=1}^{|U_i|} r_{u,i} / |U_i|$ and $\bar{r}_{N,i} = \sum_{u=1}^{|U_i|} \bar{r}_u / |U_i|$, where user $u \in U_i$, $r_{u,i}$ is u 's rating on item i , $\bar{r}_u$ is u 's average rating on all items, and $|U_i|$ is the number of users in set U_i . Node N has a distinct average rating $\bar{r}_{N,i}$ for each item i according to the set of users in U that have rated i .

The neighbourhood-based CF component takes two inputs: a code value and a target item i , and the component outputs an **approximate result**: all the nodes in code value c that have rated item i . Using these nodes, item i 's rating can be predicted.

C. Experimental Evaluation

Experimental platform and datasets: We ran all the experiments on a Linux based workstation with four Intel(R) Xeon(R) processors with a speed of 2.50 GHz and 8GB memory. The MovieLens 1M dataset [19] with 6040 users, 3953 items and over 1 million ratings was tested. We randomly selected 20% users for as test users. For each test user, we withhold a randomly selected set of 20% ratings to form a test set, and tried to predict values of these ratings based on this user's other observed ratings. Thus, there are 1208 test sets/users, where users' test ratings range from 4 to 370, and observed ratings range from 16 to 1480. In experiment, five code values at depth 1 to 5 of the R-tree were used to produce five values of approximate results ar_1 to ar_5 .

Assessment metric: We use the root-mean-square error (RMSE) [8], a weighted average error, to measure the prediction accuracy of a user u 's test set T : $RMSE = \sqrt{\frac{\sum_{i \in T} (p(u,i) - r_{u,i})^2}{n_T}}$, where n_T represents the number of items in set T , $p(u,i)$ is item i 's predicted rating and $r_{u,i}$ is its actual rating. Since different users' test sets may have different numerical scales of RMSE values, we employ the **relative error (RE)** to measure the quality of an approximate prediction. An approximate prediction ap_j 's RE RE_j is the difference between $RMSE_{ap_j}$ and $RMSE_{ep}$ divided by the $RMSE_{ep}$ [20]: $RE_j = \frac{RMSE_{ap_j} - RMSE_{ep}}{RMSE_{ep}}$. Since $RMSE_{ep}$ is fixed for each test set, a lower value of RE_j denotes higher prediction accuracy, i.e., better quality of prediction ar_j .

Experiment results: For each test set, five approximate results ar_1 to ar_5 were produced using codes c_1 to c_5 . By testing the 1,208 test sets in the *MovieLens* dataset, Figure 4(a) displays the distributions of the REs for five results using the five basic descriptive statistics in the box plot. Experiment results display that most of REs range between -0.10 to 0.10 for each result in both datasets. This means both the mean and variance values of these REs are very small. Furthermore, Figure 4(b) shows that the REs of the five approximate results ar_1 to ar_5 gradually reduce to 0. This verifies that as the information gain of results ar_1 to ar_5 increases, these results stepwise approximate the optimal result.

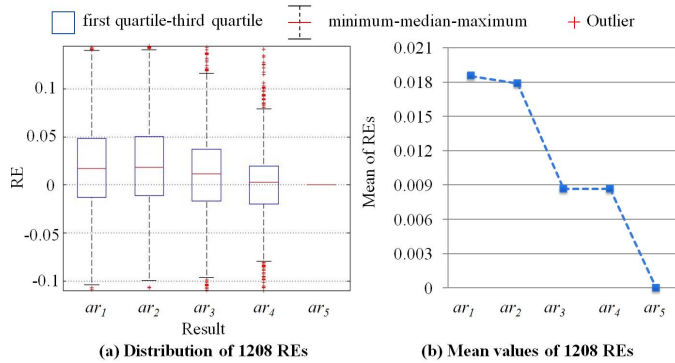


Fig. 4. Distribution and average REs on the *MovieLens* dataset.

VI. CONCLUSIONS

In this paper, we have proposed a generic approach to guarantee quality monotonicity for big data mining in a "Pay-as-You-Go" computing fashion. This quality monotonicity enables an elastic mining algorithm to produce approximate results whose quality, based on some metric, improves as the allocated time budget increases. We presented a framework for developing such algorithms using two components: coding component and mining component. We also presented two detailed case studies for designing elastic kNN classification and CF algorithms to demonstrate the applicability of the framework. Our experimental results showed that the two

elastic algorithms indeed guarantee the steady increase of observable quality metrics to time budget.

The framework presented in this paper provides a foundation for developing a wide class of elastic data mining algorithms that operate on large data *volumes*. Its successful application requires employ appropriate coding components that satisfy entropy-monotonicity. Our future work focuses on investigating other potential coding components including clustering methods such as k-means, and projection techniques such as Locality-sensitive hashing (LSH).

REFERENCES

- [1] N. A. Syed, H. Liu, and K. K. Sung, "Handling concept drifts in incremental learning with support vector machines," 1999, pp. 317-321.
- [2] P. E. Utgoff, "Incremental induction of decision trees," *Machine Learning*, vol. 4, pp. 161-186, 1989.
- [3] J. Wu, H. Xiong, and J. Chen, "Sail: summation-based incremental learning for information-theoretic clustering," in *Proceeding of the 14th ACM SIGKDD international conference on Knowledge discovery and data mining*, 2008, pp. 740-748.
- [4] S. Zilberstein, "Using anytime algorithms in intelligent systems," *AI magazine*, vol. 17, pp. 73-83, 1996.
- [5] P. Kranen, I. Assent, C. Baldauf, and T. Seidl, "Self-adaptive anytime stream clustering," 2009, pp. 249-258.
- [6] K. Ueno, X. Xi, E. Keogh, and D. J. Lee, "Anytime classification using the nearest neighbor algorithm with applications to stream mining," in *the Sixth International Conference on Data Mining (ICDM'06)*, 2006, pp. 623-632.
- [7] T. Cover and P. Hart, "Nearest neighbor pattern classification," *Information Theory, IEEE Transactions on*, vol. 13, pp. 21-27, 1967.
- [8] X. Su and T. M. Khoshgoftaar, "A survey of collaborative filtering techniques," *Advances in Artificial Intelligence*, vol. 2009, p. 4, 2009.
- [9] A. Guttman, "R-trees: a dynamic index structure for spatial searching," 1984, vol. 14.
- [10] J. R. Quinlan, "Induction of decision trees," *Machine learning*, vol. 1, pp. 81-106, 1986.
- [11] J. Han and M. Kamber, *Data mining: concepts and techniques*: Morgan Kaufmann, 2006.
- [12] M. Bern, "Approximate closest-point queries in high dimensions," *Information Processing Letters*, vol. 45, pp. 95-99, 1993.
- [13] P. Ram and A. G. Gray, "Maximum inner-product search using cone trees," in *Proceedings of the 18th ACM SIGKDD international conference on Knowledge discovery and data mining*, 2012, pp. 931-939.
- [14] K. L. Cheung and A. W. C. Fu, "Enhanced nearest neighbour search on the R-tree," *ACM SIGMOD Record*, vol. 27, pp. 16-21, 1998.
- [15] A. R. Webb, K. D. Copesey, and G. Cawley, *Statistical pattern recognition*: Wiley, 2011.
- [16] A. Frank and A. Asuncion. (2010). UCI Machine Learning Repository [http://archive.ics.uci.edu/ml. [Online].
- [17] Y. Koren, R. Bell, and C. Volinsky, "Matrix factorization techniques for recommender systems," *Computer*, vol. 42, pp. 30-37, 2009.
- [18] L. G. B. Renderer. (2012). Available: http://www.luxrender.net.[Online].
- [19] B. N. Miller, I. Albert, S. K. Lam, J. A. Konstan, and J. Riedl, "MovieLens unplugged: experiences with an occasionally connected recommender system," in *Proceedings of the 8th international conference on Intelligent user interfaces*, 2003, pp. 263-266.
- [20] M. E. Abramowitz, *Handbook of mathematical functions: with formulas, graphs, and mathematical tables*, vol. 55: Courier Dover Publications, 1964.